

Bidirectional RNNs | BiLSTM | BiGRU

1 Intro - What is a Bidirectional RNN?

In a standard RNN (or LSTM/GRU), the network only processes the sequence **forward in time**:

$$x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow \dots \rightarrow x_T$$

But sometimes, the **future context** is also important.

Example: In a sentence, knowing the word after a word can help better predict or understand it.

Bidirectional RNNs solve this by processing the sequence **both forward and backward**:

- **Forward RNN**: learns from past $\rightarrow$ future
- **Backward RNN**: learns from future $\rightarrow$ past

The outputs of both directions are then **combined** (concatenated, added, or averaged) to make predictions.

2 Why Use Bidirectional RNNs?

Advantages:

- Better context understanding (past + future)
- Improves performance in **language tasks**, **speech recognition**, **sequence tagging**, etc.
- Captures long-term dependencies more effectively

Drawbacks:

- Computationally heavier (twice the parameters for forward + backward layers)
 - Can't be used in real-time streaming where future data isn't available
-

3 Bidirectional RNN Architecture

Input sequence: $x_1, x_2, x_3, \dots, x_T$

Forward RNN: $x_1 \rightarrow h_{1_f} \rightarrow h_{2_f} \rightarrow \dots \rightarrow h_{T_f}$

Backward RNN: $x_T \rightarrow h_{T_b} \rightarrow h_{T-1_b} \rightarrow \dots \rightarrow h_{1_b}$

Output: $\text{Concatenate}(h_{t_f}, h_{t_b}) \rightarrow \text{next layer or Dense}$

- h_{t_f} = hidden state at time t in forward RNN
- h_{t_b} = hidden state at time t in backward RNN

This ensures each output considers **both past and future**.

4 Bidirectional LSTM / GRU

The same concept applies to **LSTMs** and **GRUs**:

- **BiLSTM** = LSTM layer processing forward + backward
- **BiGRU** = GRU layer processing forward + backward

Example:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Bidirectional, LSTM, Dense

model = Sequential([
    Bidirectional(LSTM(128, return_sequences=True), input_shape=(timesteps, features)),
    Dense(10, activation='softmax')
])
model.summary()
```

- `return_sequences=True` is important if stacking layers.
- `Bidirectional()` wraps any recurrent layer to make it bidirectional.

5 Demo Code Conceptually

Single BiLSTM Layer:

```
from tensorflow.keras.layers import Bidirectional, LSTM

bi_lstm = Bidirectional(LSTM(64))
```

Stacked BiLSTM:

```
model = Sequential([
    Bidirectional(LSTM(128, return_sequences=True)),
    Bidirectional(LSTM(64)),
    Dense(10, activation='softmax')
])
```

6 Applications

- **Natural Language Processing (NLP):** Part-of-speech tagging, Named Entity Recognition
- **Machine Translation:** Source and target sequences
- **Speech Recognition:** Capturing phonemes using past and future context
- **Time-Series Analysis:** Stock prediction or sequence forecasting when whole sequence is known

7 Summary Table

Concept	Description
Bidirectional RNN	Processes sequence forward + backward
BiLSTM/BiGRU	LSTM/GRU with bidirectional processing
Advantages	Captures both past & future context, improves accuracy
Drawbacks	More computation, not suitable for online real-time tasks
Keras Implementation	Use Bidirectional() wrapper

8 Intuitive Analogy

Think of reading a book:

- **Standard RNN** = You read left to right only.
 - **Bidirectional RNN** = You read left to right **and** glance ahead to see future words.
Result: You understand the meaning **much better**.
-